claude-windows / bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f\subagents\workflows\wf_b8703ccc-78b\agent-a387b3b2c22acef14.jsonl`
- SHA-256: `09e31f9ada9ceb9a30636161586131b31db8528023eea1bc1cdf8f2808ce7f2c`
- Source modified: `2026-06-17T14:37:21+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_b8703ccc-78b`
- session_id: `bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f`

Transcript

1. user (2026-06-17T14:35:18.777Z)

```
Adversarially VERIFY this code-review finding against the ACTUAL code in undefined. Default to
is_real=false unless you can point to the exact line that proves the bug is real and reachable.
Finding: severity=major; title="end_padding value changed from 1.0 to 0.5 in legacy path";
where=tools/generate_highlights.py:189-196; why=When detect_from_paths=None (the default legacy
path), the new code extracts end_padding from config.json via
cfg.get('stitching').get('end_padding', 0.5), which returns 0.5. The old code on master passed
no end_padding argument to VideoCompiler(), which defaults to 1.0. This changes the actual
segment padding applied to generated reels, making them behaviorally different from the pre-diff
code even when no --detect-from flag is used.. Read the relevant file(s) and the tests. Is this
a REAL, reachable problem in THIS diff (not a pre-existing issue, not a hypothetical)?
```

2. assistant (2026-06-17T14:35:21.007Z)

```
I'll verify this code-review finding by examining the actual code and understanding the behavior
change. Let me start by looking at the relevant file and context.
```

3. user (2026-06-17T14:35:21.449Z)

```
1       """Batch highlight-reel runner ? orchestrates "generate highlights for these videos
2       into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.
3
4       Why this exists (lessons paid for the hard way):
5       - **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work
dir
6         first, then COPIED into the destination (atomically: temp name -> os.replace). The
7         destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a
8         Google-Drive-synced folder once lost finished reels + logs.)
9       - **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync
10        client (deleted files resurrect, partial writes sync). Generate locally; publish once.
11      - **Idempotent re-runs.** Each video's prior DB segments are cleared before processing
12        (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates
13        duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
14      - **Resumable & concurrency-friendly.** A video whose reel already exists in the
15        destination is skipped (unless --force); launching a second request never undoes the
16        first's published reels. The Gemini path also isolates its chunk workdir per video
17        (PR for output/chunks/<video_id>) so concurrent videos don't collide.
18
19      Usage:
20        python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
21            --out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
22        python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
23      """
24      from __future__ import annotations
25
```

```
    import argparse
    import logging
    import os
    import shutil
    import sys
    import traceback
    from typing import Any, Dict, List, Optional, cast

    from dotenv import load_dotenv

    sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

    from backend.config import load_config
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.results import Failure
    from backend.utils.hashing import compute_video_id

    logger = logging.getLogger("highlights_runner")

    # Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.
    _WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
    _VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")


    def _atomic_publish(local_path: str, dest_path: str) -> None:
        """Copy local_path -> dest_path so the destination never sees a partial file.

        Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
        anything else in the destination folder.
        """
        os.makedirs(os.path.dirname(dest_path), exist_ok=True)
        tmp = dest_path + ".publishing.tmp"
        shutil.copy2(local_path, tmp)
        os.replace(tmp, dest_path)


    def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                                  local_work_dir: str, db_path: str, force: bool = False,
                                  skip_ai_handoff: bool = False, wasb_stream: bool = False)
-> dict:
        """Generate one highlight reel per input video into out_dir. Returns a summary dict.

        Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.

        skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini
handoff);
            candidate windows become the rallies. wasb_stream: have the WASB detector decode
the
            video on the fly instead of dumping ~46k PNGs (output-preserving fast path,
#178).
        """
        os.makedirs(out_dir, exist_ok=True)
        os.makedirs(local_work_dir, exist_ok=True)
        needs_local = segmenter in _WSL_DETECTORS

        # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is
invoked
        # standalone ? backend.main does this, but this module is its own entrypoint.
Without it the
        # AI segmenter finds no API key and silently produces zero rallies. The fully-local
path
        # (skip_ai_handoff) needs no key, so a missing .env is harmless there.
        load_dotenv()
```

```
84
85          cfg = load_config()
86          # Per-run overrides (typed config; the segmenter reads these via its dict-compat
.get).
87          # Logged by the segmenter itself once logging is configured below.
88          if skip_ai_handoff:
89              cfg.indexing.skip_ai_handoff = True
90          if wasb_stream:
91              cfg.indexing.wasb.stream_video = True
92          db = Database(db_path)
93          seg_cls = segmenter_registry.get(segmenter)
94          if seg_cls is None:
95              raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")
96
97          # Combined batch log (local; published at the end).
98          fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s",
"%H:%M:%S")
99          if not logging.getLogger().handlers:
100             logging.basicConfig(level=logging.INFO,
handlers=[logging.StreamHandler(sys.stdout)])
101             for h in logging.getLogger().handlers:
102                 h.setFormatter(fmt)
103         batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
104         batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
105         batch_fh.setFormatter(fmt)
106         logging.getLogger().addHandler(batch_fh)
107
108         summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [],
"failed": []}
109         logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
110                     len(video_paths), out_dir, segmenter, force)
111
112         for i, src in enumerate(video_paths, 1):
113             name = os.path.basename(src)
114             stem = os.path.splitext(name)[0]
115             reel_name = f"{stem} - Highlights.mp4"
116             dest_reel = os.path.join(out_dir, reel_name)
117             local_reel = os.path.join(local_work_dir, reel_name)
118
119             if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
120                 logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
121                 summary["skipped"].append(name)
122                 continue
123             if not os.path.exists(src):
124                 logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths),
name)
125                 summary["failed"].append(name)
126                 continue
127
128             run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
129             run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
130             run_fh.setFormatter(fmt)
131             logging.getLogger().addHandler(run_fh)
132             staged_src: Optional[str] = None
133             try:
134                 logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
135                 if needs_local:
136                     staged_src = os.path.join(local_work_dir, name)
137                     if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
138                         logger.info("localizing source (%.0f MB)...", os.path.getsize(src) /
1024 / 1024)
139                         shutil.copy2(src, staged_src)
```

```
                    proc_src = staged_src
                else:
                    proc_src = src

                vid = compute_video_id(proc_src)
                db.clear_video_data(vid)          # idempotent: no cross-run accumulation
                db.add_video(vid, proc_src, "processed", [])
                logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

                # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
                # main.py); cast keeps the type checker honest about that contract.
                result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
                if isinstance(result, Failure):
                    logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
                    summary["failed"].append(name)
                    continue

                segs = db.query_play_segments(vid, {})
                pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
                logger.info("rallies detected: %d", len(pairs))
                if not pairs:
                    logger.warning("no rallies -> no reel")
                    summary["no_rallies"].append(name)
                    continue

                # Compile locally, then publish atomically into the destination.
                VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
                _atomic_publish(local_reel, dest_reel)
                try:
                    os.remove(local_reel)
                except OSError:
                    pass
                logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
                summary["done"].append(name)
            except Exception:
                logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
                summary["failed"].append(name)
            finally:
                logging.getLogger().removeHandler(run_fh)
                run_fh.close()
                # publish this video's run.log (additive; never deletes anything in out_dir)
                try:
                    _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} -
run.log"))
                except OSError:
                    pass
                if staged_src and os.path.exists(staged_src):
                    try:
                        os.remove(staged_src)
                    except OSError:
                        pass

        logging.getLogger().removeHandler(batch_fh)
        batch_fh.close()
        try:
            _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
        except OSError:
            pass
        logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
        return summary


    def _resolve_videos(args) -> List[str]:
        if args.video:
```

```
203             return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v
in args.video]
204         if args.videos_dir:
205             return sorted(
206                 os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
207                 if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir,
f))
208             )
209         raise SystemExit("provide --videos-dir or one/more --video")
210
211
212     def main() -> None:
213         ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
214         ap.add_argument("--videos-dir", help="folder of source videos (all videos in it,
unless --video given)")
215         ap.add_argument("--video", action="append", help="explicit source video
(repeatable)")
216         ap.add_argument("--out", required=True, help="destination folder for reels +
run.logs")
217         ap.add_argument("--segmenter", default="gemini", help="segmenter registry key
(default: gemini)")
218         ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
219                         help="local staging dir (reels compiled here, then published)")
220         ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))
221         ap.add_argument("--force", action="store_true", help="regenerate even if a reel
already exists")
222         ap.add_argument("--skip-ai-handoff", action="store_true",
223                         help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate
windows as "
224                              "rallies with NO Gemini handoff (no API key / upload). Ignored
by --segmenter gemini.")
225         ap.add_argument("--wasb-stream", action="store_true",
226                         help="WASB decode-on-the-fly fast path (#178; output-preserving),
instead of "
227                              "dumping ~46k PNGs. Only affects wasb_hybrid.")
228         args = ap.parse_args()
229         generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
230                                   local_work_dir=args.local_work, db_path=args.db,
force=args.force,
231                                   skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)
232
233
234     if __name__ == "__main__":
235         main()
236
```

4. user (2026-06-17T14:35:21.779Z)

```
Exit code 1
/usr/bin/bash: line 1: cd: C:UsersaviduProjectsbadminton-highlight-indexer: No such file or
directory
```

5. assistant (2026-06-17T14:35:23.304Z)

Let me use the correct path syntax and search for the relevant code:

6. user (2026-06-17T14:35:23.840Z)

```
Exit code 1
/usr/bin/bash: line 1: cd: too many arguments
```

7. user (2026-06-17T14:35:24.117Z)

```
Exit code 1
/usr/bin/bash: line 1: cd: too many arguments
```

8. assistant (2026-06-17T14:35:25.999Z)

```
Let me use PowerShell instead:
```

9. user (2026-06-17T14:35:27.456Z)

```
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
8c68143 feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone (#201)
2eaccd5 feat(eval): nightly rally-quality regression tripwire (golden corpus, CPU) (#202)
34b9caa feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding
(#200)
7e8f88f feat(eval): R2 ? task-faithful tolerance-match rally metric (augments tIoU) (#199)
cf8973e docs(eval): R2 design ? task-faithful tolerance-match rally metric (FOR REVIEW) (#198)
369e90c docs(eval): golden-labeled video registry (the growing golden-finalized set) (#197)
c483346 docs(quality): real-footage validation capstone + first ground-truth at-par + dual-eval-
set discipline (#196)
bb4d6e2 feat(eval): Tier 1 (W1.5) ? hard-cap fallback enforces the window cap on continuously-
active trajectories (#195)
7d84319 fix(eval): golden-fixtures audit hardening ? close source_note/label leak + 16 confirmed
audit fixes (#194)
```

10. user (2026-06-17T14:35:28.699Z)

```
(PowerShell completed with no output)
```

11. assistant (2026-06-17T14:35:30.056Z)

```
The second command didn't show results. Let me check the current working tree status and look at
the diff more carefully:
```

12. user (2026-06-17T14:35:31.587Z)

```
On branch docs/golden-regression-fixtures-plan
Your branch and 'origin/docs/golden-regression-fixtures-plan' have diverged,
and have 18 and 1 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        artifacts/
        docs/KHELSUTRA_PRODUCT_OVERVIEW.md

nothing added to commit but untracked files present (use "git add" to track)
```

13. user (2026-06-17T14:35:32.849Z)

```
(PowerShell completed with no output)
```

14. assistant (2026-06-17T14:35:34.348Z)

```
There are no uncommitted changes to that file. Let me check the PR branch or look for recent
changes:
```

15. user (2026-06-17T14:35:36.089Z)

```
+ fix/runner-stitch-watermark
  fix/wasb-test-mocks-readability-preflight
  fix/wsl-unreadable-source-preflight
  remotes/origin/fix/runner-stitch-watermark
  remotes/origin/fix/wasb-test-mocks-readability-preflight
```

```
  remotes/origin/fix/wsl-unreadable-source-preflight
```

16. user (2026-06-17T14:35:37.388Z)

```
> a6adcc5 fix(runner): honor stitching watermark/padding + add --detect-from for hi-res
reels
  f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-
OFF) (#203)
  312cfbc fix(gemini): isolate chunk workdir per video (output/chunks/<video_id>) ? no cross-run
clobber
> 562308b fix(stitcher): merge overlapping/duplicate segments so a reel never stitches
the same rally twice (#180)
> 1deb917 fix(stitcher): merge overlapping/duplicate segments so a reel never stitches
the same rally twice
  a596b34 fix(tests): update WASB stage/run_predict mocks for the WSL readability
preflight (#177 follow-up) (#179)
  1c9be4c docs(platform): video-locality mental model ? the 10GB question (#106)
> 6de70aa fix(stitcher): escape drive-letter colon in ffmpeg filtergraph paths
(#108)
> 3461898 feat(api): enforce stitching.min_shot_count on /api/compile (#107)
  9c84fea docs(pmf): remove the associate paid component (owner has another structure in
mind)
  ffdfde0 Merge pull request #103 from avidullu:claude/watermark-bundled-font
> 112e75d feat(stitcher): bundle Apache-2.0 default watermark font + log fallback
reason
> 82e479f Merge pull request #102 from avidullu:fix/stitching-padding-config
> 30e4899 fix(api): thread config stitching paddings into live VideoCompiler sites
> 10864f3 feat(stitcher): configurable branding watermark on highlight reels (#101)
  020ff61 docs: status sync - PR-2 MERGED (#99, live-E2E-verified), sweep recorded,
BACKLOG shlex item closed (#100)
  34cdcfa fix(detectors): namespace WSL/cache/output artifacts by video_id, not basename (#64)
> 13d4573 fix(stitcher): restore f-string quotes dropped in logging conversion
(SyntaxError on master) (#63)
  a5477c2 docs: next-steps + multisport findings (WASB transfers to table tennis, not
pickleball) (#62)
  6c9e585 chore: update input video file path and reduce chunk overlap configuration
> 77b9973 Log estimated final video duration after post-hoc validation, factoring in
padding
  7a45321 Add metrics tracking and summary table logging for Gemini API chunked
processing
  9582752 Persist compressed videos to output/Compressed Videos instead of deleting them to
allow for instant caching
and reuse on subsequent runs
> 623214a Add begin_padding logic to stitcher and config to capture the serve perfectly
at the start of each rally
  0898b99 Update video compression
logic to dynamically calculate video
bitrate using ffprobe to accurately
target a
~1.9GB file size for videos
over 2GB, preventing extreme
degradation.
  f089691 Improve Gemini rally detection: stricter prompt for grounded shuttle + player stop +
temperature=0.0
> b9d7e19 Implement actual Gemini shot_count extraction, filter stitched
rallies based on min_shot_count in
config,
and support robust SQLite caching
to skip Gemini API when
database has rallies
> 5836e1e Implement configurable end_padding (default 1.0s)
inside VideoCompiler and config.json
to prevent abrupt
rally endings
  8f4c564 Add real-time progress
```

```
logging for MD5 hashing, add
processing milestones tracking in
Gemini segmenter, and
bypass heavy OpenCV validations on
large files
> 0b79ab6 Optimize Gemini segmenter with automatic low-bitrate compression for large videos,
disable config
debug_duration by default, and update stitcher output
filename
  0f76429 Refactor rally detection
to pluggable base and registry,
integrate Gemini 2.5 Flash
segmenter, add local
fast ffmpeg trimming, and update
developer docs
```

17. assistant (2026-06-17T14:35:39.660Z)

```
I see a recent commit "fix(runner): honor stitching watermark/padding + add --detect-from for
hi-res reels" (a6adcc5). Let me check that commit:
```

18. user (2026-06-17T14:35:41.718Z)

```
commit a6adcc57290bd65c3ca7c092d0568ff02cf60378
Author: Avi Dullu <avi.dullu@gmail.com>
Date:   Wed Jun 17 19:59:43 2026 +0530

    fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels

    The batch runner (tools/generate_highlights.py) constructed VideoCompiler bare ?
    `VideoCompiler(local_work_dir).compile_highlights(...)` ? so reels shipped UN-watermarked
    (watermark defaults to None/off) and used the ctor's begin/end padding (0.5/1.0) instead of
    config.json `stitching.{begin_padding,end_padding}` (0.5/0.5). Every other compile call site
    threads stitching config; this one didn't. Now it reads `stitching.*` dict-compatibly and
    passes watermark + paddings through (the on-by-default branding mark).

    Also add `--detect-from`: run the detector on a cheaper proxy (e.g. a 1080p downscale of a
4K
    source) while stitching the reel from the full-res --video. Timestamps transfer 1:1 (pure
    temporal-preserving downscale), so it's the bandwidth/GPU-saver split the workflow wanted ?
    detect on the proxy, deliver a 4K reel. Reel is named after the --video (the thing watched).

    Tests: extend test_generate_highlights.py ? watermark+padding are threaded into
VideoCompiler
    (regression lock), --detect-from keys detection off the proxy but stitches from the
original,
    and _resolve_detect_from validates 1:1 pairing.

    Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

 tests/test_generate_highlights.py | 89 ++++++++++++++++++++++++++++++++++++++++
 tools/generate_highlights.py      | 83 +++++++++++++++++++++++++++++++-------
 2 files changed, 156 insertions(+), 16 deletions(-)
```

19. user (2026-06-17T14:35:43.861Z)

```
commit a6adcc57290bd65c3ca7c092d0568ff02cf60378
Author: Avi Dullu <avi.dullu@gmail.com>
Date:   Wed Jun 17 19:59:43 2026 +0530

    fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels

    The batch runner (tools/generate_highlights.py) constructed VideoCompiler bare ?
    `VideoCompiler(local_work_dir).compile_highlights(...)` ? so reels shipped UN-watermarked
```

```
    (watermark defaults to None/off) and used the ctor's begin/end padding (0.5/1.0) instead of
    config.json `stitching.{begin_padding,end_padding}` (0.5/0.5). Every other compile call site
    threads stitching config; this one didn't. Now it reads `stitching.*` dict-compatibly and
    passes watermark + paddings through (the on-by-default branding mark).

    Also add `--detect-from`: run the detector on a cheaper proxy (e.g. a 1080p downscale of a
4K
    source) while stitching the reel from the full-res --video. Timestamps transfer 1:1 (pure
    temporal-preserving downscale), so it's the bandwidth/GPU-saver split the workflow wanted ?
    detect on the proxy, deliver a 4K reel. Reel is named after the --video (the thing watched).

    Tests: extend test_generate_highlights.py ? watermark+padding are threaded into
VideoCompiler
    (regression lock), --detect-from keys detection off the proxy but stitches from the
original,
    and _resolve_detect_from validates 1:1 pairing.

    Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

diff --git a/tools/generate_highlights.py b/tools/generate_highlights.py
index ae9f045..565663a 100644
--- a/tools/generate_highlights.py
+++ b/tools/generate_highlights.py
@@ -63,7 +63,8 @@ def _atomic_publish(local_path: str, dest_path: str) -> None:

 def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                               local_work_dir: str, db_path: str, force: bool = False,
-                              skip_ai_handoff: bool = False, wasb_stream: bool = False) ->
dict:
+                              skip_ai_handoff: bool = False, wasb_stream: bool = False,
+                              detect_from_paths: Optional[List[str]] = None) -> dict:
     """Generate one highlight reel per input video into out_dir. Returns a summary dict.

     Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
@@ -71,6 +72,11 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
     skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
         candidate windows become the rallies. wasb_stream: have the WASB detector decode the
         video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).
+    detect_from_paths: optional list parallel to video_paths. When given, the detector runs on
+        detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source) but the
+        reel is stitched from video_paths[i] at full resolution. Rally timestamps transfer 1:1
+        (a pure temporal-preserving downscale shares the timeline). None = detect+stitch the
+        same source (legacy behaviour).
     """
     os.makedirs(out_dir, exist_ok=True)
     os.makedirs(local_work_dir, exist_ok=True)
@@ -115,13 +121,17 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
         reel_name = f"{stem} - Highlights.mp4"
         dest_reel = os.path.join(out_dir, reel_name)
         local_reel = os.path.join(local_work_dir, reel_name)
+        # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
+        # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
+        detect_src = detect_from_paths[i - 1] if detect_from_paths else src

         if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
             logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
             summary["skipped"].append(name)
             continue
-        if not os.path.exists(src):
-            logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
+        if not os.path.exists(src) or not os.path.exists(detect_src):
+            logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
```

```
+                          i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
              summary["failed"].append(name)
              continue

@@ -132,23 +142,29 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
          staged_src: Optional[str] = None
          try:
              logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
+             # Localize the DETECTION source for WSL detectors (the WSL FS can't read Drive/UNC
+             # via /mnt). The stitch source (`src`) is read by ffmpeg directly during compile,
so
+             # only the detector input ever needs staging.
              if needs_local:
-                 staged_src = os.path.join(local_work_dir, name)
-                 if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
-                     logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)
-                     shutil.copy2(src, staged_src)
-                 proc_src = staged_src
+                 staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
+                 if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
+                     logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
+                     shutil.copy2(detect_src, staged_src)
+                 proc_detect = staged_src
              else:
-                 proc_src = src
+                 proc_detect = detect_src

-             vid = compute_video_id(proc_src)
+             vid = compute_video_id(proc_detect)
              db.clear_video_data(vid)          # idempotent: no cross-run accumulation
-             db.add_video(vid, proc_src, "processed", [])
+             db.add_video(vid, proc_detect, "processed", [])
+             if detect_src != src:
+                 logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
+                             os.path.basename(detect_src), name)
              logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

              # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
              # main.py); cast keeps the type checker honest about that contract.
-             result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
+             result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
              if isinstance(result, Failure):
                  logger.error("segmentation FAILED: %s", getattr(result, "message", result))
                  summary["failed"].append(name)
@@ -162,8 +178,20 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
                  summary["no_rallies"].append(name)
                  continue

-             # Compile locally, then publish atomically into the destination.
-             VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
+             # Stitch source: the original `src` when detecting on a separate proxy (hi-res
reel);
+             # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
+             stitch_src = src if detect_from_paths else proc_detect
+             # Honor the configured branding watermark + segment paddings (stitching.*). Read
+             # dict-compatibly so a plain-dict config (tests) and the typed AppConfig both work.
+             # NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
```

```
+            # ctor default paddings; this threads config.json through (the on-by-default mark).
+            st = cfg.get("stitching") or {}
+            VideoCompiler(
+                local_work_dir,
+                begin_padding=float(st.get("begin_padding", 0.5)),
+                end_padding=float(st.get("end_padding", 0.5)),
+                watermark=st.get("watermark"),
+            ).compile_highlights(stitch_src, pairs, reel_name)
             _atomic_publish(local_reel, dest_reel)
             try:
                 os.remove(local_reel)
@@ -209,6 +237,21 @@ def _resolve_videos(args) -> List[str]:
     raise SystemExit("provide --videos-dir or one/more --video")


+def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
+    """Resolve --detect-from into a list parallel to video_paths (or None if unused).
+
+    Requires explicit --video (not --videos-dir) and a 1:1 count match ? each detection
+    proxy maps to exactly one --video by position."""
+    if not getattr(args, "detect_from", None):
+        return None
+    if not args.video:
+        raise SystemExit("--detect-from requires explicit --video (not --videos-dir)")
+    if len(args.detect_from) != len(args.video):
+        raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
+                         f"--video count ({len(args.video)}); pair them 1:1 in order")
+    return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
args.detect_from]
+
+
 def main() -> None:
     ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
     ap.add_argument("--videos-dir", help="folder of source videos (all videos in it, unless
--video given)")
@@ -225,10 +268,18 @@ def main() -> None:
     ap.add_argument("--wasb-stream", action="store_true",
                     help="WASB decode-on-the-fly fast path (#178; output-preserving), instead
of "
                          "dumping ~46k PNGs. Only affects wasb_hybrid.")
+    ap.add_argument("--detect-from", action="append",
+                    help="detection source ? e.g. a 1080p proxy. The detector analyzes THIS "
+                         "(cheaper to decode) while the reel is stitched from the matching
--video "
+                         "at full resolution (e.g. the 4K original). Repeat to pair 1:1 with
--video "
+                         "in the same order. Timestamps transfer 1:1 (pure temporal
downscale).")
     args = ap.parse_args()
-    generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
+    video_paths = _resolve_videos(args)
+    detect_from_paths = _resolve_detect_from(args, video_paths)
+    generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
                               local_work_dir=args.local_work, db_path=args.db,
force=args.force,
-                              skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)
+                              skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream,
+                              detect_from_paths=detect_from_paths)


 if __name__ == "__main__":
```

20. assistant (2026-06-17T14:35:45.641Z)

Perfect! Now I can see the diff. Let me examine the critical section around line 189-196 (according to the finding). The finding claims there's a behavior change in the `end_padding` value. Let me look at the VideoCompiler constructor to understand the defaults:

21. user (2026-06-17T14:35:46.540Z)

Found 1 file
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py

22. user (2026-06-17T14:35:46.682Z)

```
1       import os
2       import subprocess
3       import tempfile
4       import logging
5       from pathlib import Path
6       from typing import Any, Dict, List, Optional, Tuple, Union
7
8       logger = logging.getLogger(__name__)
9
10      # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11      # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12      # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
13      # backend/pipeline/stitcher/, so parents[2] == backend/.
14      _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16      class VideoCompiler:
17          def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                       watermark: Optional[Any] = None):
19              self.output_dir = output_dir
20              self.end_padding = end_padding
21              self.begin_padding = begin_padding
22              # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23              # normalized to a dict (or None) so this module stays config-package-agnostic.
24              self.watermark = self._normalize_watermark(watermark)
25              os.makedirs(output_dir, exist_ok=True)
26
27          @staticmethod
28          def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29              if watermark is None:
30                  return None
31              if hasattr(watermark, "model_dump"):
32                  return watermark.model_dump()
33              if isinstance(watermark, dict):
34                  return dict(watermark)
35              return None
36
37          @staticmethod
38          def _ff_quote(path: str) -> str:
39              """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40              filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41              slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43              The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44              option parser treats ':' as the option separator, so a Windows drive-letter
45              path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46              the encode fails with "Invalid argument". This silently disabled the
47              on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
```

```
48              return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
49
50          @staticmethod
51          def _parse_time(val: Union[int, float, str]) -> float:
52              """Normalizes a timestamp value to float seconds.
53              Handles: float/int (pass-through), plain numeric strings ("12.4"),
54              and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55              This mirrors the parse_time logic used in gemini.py to ensure
56              the stitcher can always consume whatever format the indexer produced."""
57              if isinstance(val, (int, float)):
58                  return float(val)
59              if isinstance(val, str):
60                  val = val.strip()
61                  if ":" in val:
62                      # Handle MM:SS, HH:MM:SS, etc.
63                      parts = val.split(":")
64                      parts.reverse()
65                      total = 0.0
66                      for i, p in enumerate(parts):
67                          try:
68                              total += float(p) * (60 ** i)
69                          except ValueError:
70                              pass
71                      return total
72                  try:
73                      return float(val)
74                  except ValueError:
75                      logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
76                      return 0.0
77              logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
78              return 0.0
79
80          @staticmethod
81          def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
float]]:
82              """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
83
84              A highlight reel must never replay the same footage. The same rally can land in
85              the segment list more than once ? e.g. two detector runs accumulated for one
86              video (add_video is an UPSERT, so re-running without a prune leaves both sets),
or
87              chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
rally
88              twice. Merging any range that overlaps or abuts the previous one collapses true
89              duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
90              Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
91              """
92              parsed = []
93              for raw_s, raw_e in segments:
94                  s = VideoCompiler._parse_time(raw_s)
95                  e = VideoCompiler._parse_time(raw_e)
96                  if e > s:
97                      parsed.append((s, e))
98              parsed.sort()
99              merged: List[Tuple[float, float]] = []
100             for s, e in parsed:
```

23. assistant (2026-06-17T14:35:48.828Z)

Great! I can see the VideoCompiler constructor at line 17. The defaults are:
- `end_padding: float = 1.0`
- `begin_padding: float = 0.5`

Now let me look at the old code path (what it was before the diff) and the new code to
understand the claim:

24. user (2026-06-17T14:35:51.955Z)

```
              segs = db.query_play_segments(vid, {})
              pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
              logger.info("rallies detected: %d", len(pairs))
              if not pairs:
                  logger.warning("no rallies -> no reel")
                  summary["no_rallies"].append(name)
                  continue

              # Compile locally, then publish atomically into the destination.
>             VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs,
reel_name)
              _atomic_publish(local_reel, dest_reel)
              try:
                  os.remove(local_reel)
              except OSError:
                  pass
              logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
              summary["done"].append(name)
          except Exception:
              logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
              summary["failed"].append(name)
```

25. user (2026-06-17T14:35:52.006Z)

```
1       {
2           "sport": "badminton",
3           "ai_provider": "gemini",
4           "ai_model": "gemini-flash-latest",
5           "validation": {
6               "min_resolution_width": 1280,
7               "min_resolution_height": 720,
8               "min_fps": 29.9,
9               "min_blur_threshold": 20.0,
10              "max_scene_cuts_per_minute": 0.5,
11              "relevance": {
12                  "court_green_lower": [
13                      35,
14                      40,
15                      40
16                  ],
17                  "court_green_upper": [
18                      85,
19                      255,
20                      255
21                  ],
22                  "court_pixels_min_ratio": 0.05
23              }
24          },
25          "indexing": {
26              "default_segmenter": "gemini",
27              "use_gpu": true,
28              "motion_threshold": 0.08,
29              "min_segment_duration": 3.0,
30              "dead_time_merge_gap": 2.0,
31              "chunk_duration": 90,
32              "chunk_overlap": 10,
33              "detector_model": "fasterrcnn_resnet50_fpn",
34              "detector_score_threshold": 0.5,
35              "yolo_velocity_threshold": 0.15,
36              "yolo_frame_skip": 3,
```

```
37              "yolo_tracker": "bytetrack.yaml",
38              "yolo_prefetch_workers": 2,
39              "min_action_window_duration": 2.0,
40              "log_yolo_candidates": true,
41              "store_yolo_candidates": true,
42              "ai_handoff_padding": 2.0,
43              "ai_max_workers": 1,
44              "tracknet": {
45                  "wsl_distro": "Ubuntu",
46                  "repo_dir": "~/models/TrackNetV4",
47                  "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48                  "conda_env": "TrackNetV4",
49                  "weights_path": "",
50                  "queue_length": 5,
51                  "stage_in_wsl": true,
52                  "wsl_stage_dir": "~/clips",
53                  "timeout_sec": 1800,
54                  "velocity_threshold": 0.02
55              },
56              "wasb": {
57                  "wsl_distro": "Ubuntu",
58                  "repo_dir": "~/models/WASB-SBDT",
59                  "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60                  "conda_env": "wasb",
61                  "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
62                  "sport": "badminton",
63                  "wsl_stage_dir": "~/clips",
64                  "timeout_sec": 1800,
65                  "velocity_threshold": 0.02
66              }
67          },
68          "output": {
69              "default_highlights_name": "highlights.mp4",
70              "db_name": "sports_indexer.db"
71          },
72          "stitching": {
73              "begin_padding": 0.5,
74              "end_padding": 0.5,
75              "use_cache": false,
76              "min_shot_count": 1,
77              "watermark": {
78                  "enabled": true,
79                  "text": "Generated by Khelsutra.guru",
80                  "position": "bottom-right",
81                  "opacity": 0.85,
82                  "logo_path": "",
83                  "font_path": ""
84              }
85          }
86      }
```

26. assistant (2026-06-17T14:35:57.506Z)

Perfect! Now I have all the information. Let me trace through the logic carefully:

OLD CODE (before the diff):
- Line 166 (before): `VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)`
- This calls VideoCompiler with just `output_dir`, using all defaults
- VideoCompiler defaults: `end_padding=1.0`, `begin_padding=0.5`
- **So the old behavior used `end_padding=1.0`**

NEW CODE (after the diff):
Looking at lines 189-196 in the diff:

```python
st = cfg.get("stitching") or {}
VideoCompiler(
    local_work_dir,
    begin_padding=float(st.get("begin_padding", 0.5)),
    end_padding=float(st.get("end_padding", 0.5)),
    watermark=st.get("watermark"),
).compile_highlights(stitch_src, pairs, reel_name)
```

Now the critical question: **In the default legacy path (when `detect_from_paths=None`), what value does `end_padding` get?**

- The code reads: `st.get("end_padding", 0.5)`
- `st` is `cfg.get("stitching") or {}`
- Looking at config.json line 74: `"end_padding": 0.5`
- So when using the config file, `st.get("end_padding", 0.5)` returns **0.5**

The finding states: "the new code extracts end_padding from config.json via cfg.get('stitching').get('end_padding', 0.5), which returns 0.5."

But wait - let me check if this code path is actually used in the legacy scenario. Let me look at the code path more carefully:

27. user (2026-06-17T14:35:59.584Z)

```
  def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                                local_work_dir: str, db_path: str, force: bool = False,
                                skip_ai_handoff: bool = False, wasb_stream: bool = False,
>                               detect_from_paths: Optional[List[str]] = None) ->
dict:
      """Generate one highlight reel per input video into out_dir. Returns a summary
dict.

      Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never
deletes.

      skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini
handoff);
          candidate windows become the rallies. wasb_stream: have the WASB detector decode the
          video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).
>     detect_from_paths: optional list parallel to video_paths. When given, the detector
runs on
>         detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source)
but the
          reel is stitched from video_paths[i] at full resolution. Rally timestamps
transfer 1:1
          (a pure temporal-preserving downscale shares the timeline). None =
detect+stitch the
          same source (legacy behaviour).
      """
      os.makedirs(out_dir, exist_ok=True)
          reel_name = f"{stem} - Highlights.mp4"
          dest_reel = os.path.join(out_dir, reel_name)
          local_reel = os.path.join(local_work_dir, reel_name)
          # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
          # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
>         detect_src = detect_from_paths[i - 1] if detect_from_paths else src

          if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) >
0:
              logger.info("[%d/%d] SKIP %s ? reel already published", i,
len(video_paths), name)
              summary["skipped"].append(name)
              continue
```

```
                    summary["no_rallies"].append(name)
                    continue

                # Stitch source: the original `src` when detecting on a separate proxy (hi-res
reel);
                # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
>               stitch_src = src if detect_from_paths else proc_detect
                # Honor the configured branding watermark + segment paddings
(stitching.*). Read
                # dict-compatibly so a plain-dict config (tests) and the typed AppConfig
both work.
                # NOTE: the runner previously dropped these ? reels shipped UN-watermarked
with the
                # ctor default paddings; this threads config.json through (the on-by-
default mark).
                st = cfg.get("stitching") or {}
                VideoCompiler(
                    local_work_dir,
>                   begin_padding=float(st.get("begin_padding", 0.5)),
>                   end_padding=float(st.get("end_padding", 0.5)),
>                   watermark=st.get("watermark"),
                ).compile_highlights(stitch_src, pairs, reel_name)
                _atomic_publish(local_reel, dest_reel)
                try:
                    os.remove(local_reel)
                except OSError:
                            "(cheaper to decode) while the reel is stitched from the matching
--video "
                            "at full resolution (e.g. the 4K original). Repeat to pair 1:1 with
--video "
                            "in the same order. Timestamps transfer 1:1 (pure temporal
downscale).")
      args = ap.parse_args()
      video_paths = _resolve_videos(args)
>     detect_from_paths = _resolve_detect_from(args, video_paths)
      generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
                                local_work_dir=args.local_work, db_path=args.db,
force=args.force,
                                skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream,
>                               detect_from_paths=detect_from_paths)


  if __name__ == "__main__":
      main()
```

28. assistant (2026-06-17T14:36:04.181Z)

Good. Now let me check if there's a scenario where the code path is actually taken. The key question is: **Is the code that reads from config.json always executed regardless of detect_from_paths?**

Looking at the new code:
```python
st = cfg.get("stitching") or {}
VideoCompiler(
    local_work_dir,
    begin_padding=float(st.get("begin_padding", 0.5)),
    end_padding=float(st.get("end_padding", 0.5)),
    watermark=st.get("watermark"),
).compile_highlights(stitch_src, pairs, reel_name)
```

YES - this code is always executed. It's not conditional on `detect_from_paths`. So when

`detect_from_paths=None` (the legacy/default path), the code:
1. Reads from `cfg` (which loads config.json at line 85)
2. Extracts `st = cfg.get("stitching") or {}`
3. Calls `st.get("end_padding", 0.5)` which returns `0.5` (from config.json line 74)

So the finding is **CORRECT**. The behavior HAS changed:

OLD behavior (before the diff):
- Legacy path (no --detect-from): `VideoCompiler(local_work_dir)` ? uses `end_padding=1.0` (the constructor default)

NEW behavior (after the diff):
- Legacy path (no --detect-from): `st.get("end_padding", 0.5)` ? uses `end_padding=0.5` (from config.json)

Let me verify this is a real issue by looking at the tests to see if they catch it:

29. user (2026-06-17T14:36:04.422Z)

```
1       """tools/generate_highlights.py ? the batch highlight runner. The load-bearing
guarantees
2       are NON-DESTRUCTIVE + atomic publish + idempotent skip; those are what we lock here
3       (heavy deps ? segmenter/compiler/db ? are mocked)."""
4       import os
5       from unittest.mock import MagicMock, patch
6
7       import tools.generate_highlights as gh
8
9
10      def test_atomic_publish_is_nondestructive(tmp_path):
11          src = tmp_path / "local_reel.mp4"
12          src.write_bytes(b"reel-bytes")
13          dest_dir = tmp_path / "out"
14          dest_dir.mkdir()
15          sibling = dest_dir / "Someone Else - Highlights.mp4"
16          sibling.write_bytes(b"do-not-touch")
17
18          gh._atomic_publish(str(src), str(dest_dir / "Me - Highlights.mp4"))
19
20          assert (dest_dir / "Me - Highlights.mp4").read_bytes() == b"reel-bytes"
21          assert sibling.read_bytes() == b"do-not-touch"          # sibling untouched
22          assert not list(dest_dir.glob("*.publishing.tmp"))      # no partial left behind
23
24
25      def test_resolve_videos_explicit_and_dir(tmp_path):
26          (tmp_path / "a.MP4").write_bytes(b"x")
27          (tmp_path / "b.mp4").write_bytes(b"x")
28          (tmp_path / "notes.txt").write_text("ignore")
29          args = MagicMock(video=None, videos_dir=str(tmp_path))
30          found = gh._resolve_videos(args)
31          assert [os.path.basename(p) for p in found] == ["a.MP4", "b.mp4"]   # videos only,
sorted
32          args2 = MagicMock(video=["only.MP4"], videos_dir=str(tmp_path))
33          assert gh._resolve_videos(args2) == [os.path.join(str(tmp_path), "only.MP4")]
34
35
36      def test_batch_skips_existing_reel_and_preserves_other_work(tmp_path):
37          """force=False: a video whose reel already exists is skipped, the segmenter is never
38          called, and unrelated files already in the destination are left intact."""
39          out = tmp_path / "out"
40          out.mkdir()
41          (out / "Adarsh v Avi - Highlights.mp4").write_bytes(b"already-published")
42          (out / "Adarsh v Avi - run.log").write_text("prior log")
43          other = out / "Someone Else - Highlights.mp4"
44          other.write_bytes(b"untouched")
```

```
45          src = tmp_path / "Adarsh v Avi.MP4"
46          src.write_bytes(b"src")
47
48          seg_registry = MagicMock()
49          with patch.object(gh, "load_config", return_value={}), \
50               patch.object(gh, "Database", return_value=MagicMock()), \
51               patch.object(gh, "segmenter_registry", seg_registry):
52              summary = gh.generate_highlights_batch(
53                  [str(src)], str(out), segmenter="gemini",
54                  local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
force=False)
55
56          assert summary["skipped"] == ["Adarsh v Avi.MP4"]
57          # never processed: the resolved segmenter class was never instantiated/run
58          seg_registry.get.return_value.assert_not_called()
59          assert other.read_bytes() == b"untouched"            # other work preserved
60          assert (out / "Adarsh v Avi - Highlights.mp4").read_bytes() == b"already-published"
# not regenerated
61
62
63      def test_batch_applies_local_overrides(tmp_path):
64          """--skip-ai-handoff / --wasb-stream inject into the typed config before
segmentation."""
65          from backend.config import AppConfig
66          cfg = AppConfig()
67          assert cfg.indexing.skip_ai_handoff is False          # default off
68          assert cfg.indexing.wasb.stream_video is False
69
70          out = tmp_path / "out"
71          out.mkdir()
72          src = tmp_path / "clip.MP4"
73          src.write_bytes(b"src")
74
75          seg_instance = MagicMock()
76          seg_instance.process_video.return_value = []          # no rallies -> no
compile/publish
77          seg_registry = MagicMock()
78          seg_registry.get.return_value = MagicMock(return_value=seg_instance)
79
80          with patch.object(gh, "load_config", return_value=cfg), \
81               patch.object(gh, "Database", return_value=MagicMock()), \
82               patch.object(gh, "compute_video_id", return_value="vid123"), \
83               patch.object(gh, "segmenter_registry", seg_registry):
84              gh.generate_highlights_batch(
85                  [str(src)], str(out), segmenter="gemini",
86                  local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
87                  force=True, skip_ai_handoff=True, wasb_stream=True)
88
89          assert cfg.indexing.skip_ai_handoff is True
90          assert cfg.indexing.wasb.stream_video is True
91
```

30. assistant (2026-06-17T14:36:05.807Z)

Let me check if there are newer tests added in the commit:

31. user (2026-06-17T14:36:08.816Z)

```
Exit code 1
tail: The term 'tail' is not recognized as a name of a cmdlet, function, script
file, or executable program.
Check the spelling of the name, or if a path was included, verify that the path is
correct and try again.
```

32. user (2026-06-17T14:36:11.420Z)

```
def test_atomic_publish_is_nondestructive(tmp_path):
    src = tmp_path / "local_reel.mp4"
    src.write_bytes(b"reel-bytes")
    dest_dir = tmp_path / "out"
    dest_dir.mkdir()
    sibling = dest_dir / "Someone Else - Highlights.mp4"
    sibling.write_bytes(b"do-not-touch")

    gh._atomic_publish(str(src), str(dest_dir / "Me - Highlights.mp4"))

    assert (dest_dir / "Me - Highlights.mp4").read_bytes() == b"reel-bytes"
    assert sibling.read_bytes() == b"do-not-touch"          # sibling untouched
    assert not list(dest_dir.glob("*.publishing.tmp"))      # no partial left behind


def test_resolve_videos_explicit_and_dir(tmp_path):
    (tmp_path / "a.MP4").write_bytes(b"x")
    (tmp_path / "b.mp4").write_bytes(b"x")
    (tmp_path / "notes.txt").write_text("ignore")
    args = MagicMock(video=None, videos_dir=str(tmp_path))
    found = gh._resolve_videos(args)
    assert [os.path.basename(p) for p in found] == ["a.MP4", "b.mp4"]   # videos only, sorted
    args2 = MagicMock(video=["only.MP4"], videos_dir=str(tmp_path))
    assert gh._resolve_videos(args2) == [os.path.join(str(tmp_path), "only.MP4")]


def test_batch_skips_existing_reel_and_preserves_other_work(tmp_path):
    """force=False: a video whose reel already exists is skipped, the segmenter is never
    called, and unrelated files already in the destination are left intact."""
    out = tmp_path / "out"
    out.mkdir()
    (out / "Adarsh v Avi - Highlights.mp4").write_bytes(b"already-published")
    (out / "Adarsh v Avi - run.log").write_text("prior log")
    other = out / "Someone Else - Highlights.mp4"
    other.write_bytes(b"untouched")
    src = tmp_path / "Adarsh v Avi.MP4"
    src.write_bytes(b"src")

    seg_registry = MagicMock()
    with patch.object(gh, "load_config", return_value={}), \
         patch.object(gh, "Database", return_value=MagicMock()), \
         patch.object(gh, "segmenter_registry", seg_registry):
        summary = gh.generate_highlights_batch(
            [str(src)], str(out), segmenter="gemini",
            local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
force=False)

    assert summary["skipped"] == ["Adarsh v Avi.MP4"]
    # never processed: the resolved segmenter class was never instantiated/run
    seg_registry.get.return_value.assert_not_called()
    assert other.read_bytes() == b"untouched"            # other work preserved
    assert (out / "Adarsh v Avi - Highlights.mp4").read_bytes() == b"already-published"  # not
regenerated


def test_batch_applies_local_overrides(tmp_path):
    """--skip-ai-handoff / --wasb-stream inject into the typed config before segmentation."""
    from backend.config import AppConfig
    cfg = AppConfig()
    assert cfg.indexing.skip_ai_handoff is False          # default off
    assert cfg.indexing.wasb.stream_video is False

    out = tmp_path / "out"
```

```
    out.mkdir()
    src = tmp_path / "clip.MP4"
    src.write_bytes(b"src")

    seg_instance = MagicMock()
    seg_instance.process_video.return_value = []          # no rallies -> no compile/publish
    seg_registry = MagicMock()
    seg_registry.get.return_value = MagicMock(return_value=seg_instance)

    with patch.object(gh, "load_config", return_value=cfg), \
         patch.object(gh, "Database", return_value=MagicMock()), \
         patch.object(gh, "compute_video_id", return_value="vid123"), \
         patch.object(gh, "segmenter_registry", seg_registry):
        gh.generate_highlights_batch(
            [str(src)], str(out), segmenter="gemini",
            local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
            force=True, skip_ai_handoff=True, wasb_stream=True)

    assert cfg.indexing.skip_ai_handoff is True
    assert cfg.indexing.wasb.stream_video is True


def _run_one(tmp_path, *, video_paths, detect_from_paths=None, segmenter="gemini"):
    """Drive generate_highlights_batch once with a fake compiler/segmenter/db, returning the
    captured VideoCompiler ctor+compile args and the compute_video_id mock."""
    from backend.config import AppConfig
    cfg = AppConfig()                                     # watermark on by default
    out = tmp_path / "out"
    out.mkdir()
    db = MagicMock()
    db.query_play_segments.return_value = [{"start_time": 1.0, "end_time": 5.0}]
    seg_instance = MagicMock()
    seg_instance.process_video.return_value = ["ok"]    # non-Failure
    seg_registry = MagicMock()
    seg_registry.get.return_value = MagicMock(return_value=seg_instance)
    captured: dict = {}
    cvid = MagicMock(return_value="vid123")
    with patch.object(gh, "load_config", return_value=cfg), \
         patch.object(gh, "Database", return_value=db), \
         patch.object(gh, "compute_video_id", cvid), \
         patch.object(gh, "VideoCompiler", _fake_compiler(captured)), \
         patch.object(gh, "segmenter_registry", seg_registry):
        gh.generate_highlights_batch(
            [str(p) for p in video_paths], str(out), segmenter=segmenter,
            local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
            force=True, detect_from_paths=[str(p) for p in detect_from_paths] if
detect_from_paths else None)
    return captured, cvid, out


def test_batch_threads_watermark_and_padding_into_compiler(tmp_path):
    """REGRESSION: the runner must pass stitching.watermark + begin/end_padding to VideoCompiler
    (it previously constructed VideoCompiler bare -> reels shipped un-watermarked)."""
    src = tmp_path / "clip.MP4"
    src.write_bytes(b"src")
    captured, _cvid, out = _run_one(tmp_path, video_paths=[src])

    wm = captured["ctor"]["watermark"]
    assert wm and wm.get("enabled") is True                # on-by-default mark threaded
through
    assert wm.get("text") == "Generated by Khelsutra.guru"
    assert captured["ctor"]["begin_padding"] == 0.5        # config paddings, not the ctor 1.0
default
    assert captured["ctor"]["end_padding"] == 0.5
    assert captured["compile"]["raw"] == str(src)          # stitches from the (only) source
```

```
    assert (out / "clip - Highlights.mp4").exists()         # published


def test_batch_detect_from_proxy_stitches_from_original(tmp_path):
    """--detect-from: the detector runs on the proxy (video_id keyed off it) but the reel is
    stitched from the full-res --video, and named after it."""
    orig = tmp_path / "GX01.MP4"
    orig.write_bytes(b"original-4k")
    proxy = tmp_path / "GX01_1080p.mp4"
    proxy.write_bytes(b"proxy-1080")
    captured, cvid, out = _run_one(tmp_path, video_paths=[orig], detect_from_paths=[proxy])

    cvid.assert_called_once_with(str(proxy))                # detection keyed off the proxy
    assert captured["compile"]["raw"] == str(orig)          # reel stitched from the original
    assert (out / "GX01 - Highlights.mp4").exists()         # named after the original, not the
proxy


def test_resolve_detect_from_validates_pairing(tmp_path):
    assert gh._resolve_detect_from(MagicMock(detect_from=None), ["v"]) is None
    with pytest.raises(SystemExit):                         # count mismatch
        gh._resolve_detect_from(MagicMock(detect_from=["a", "b"], video=["v1"],
videos_dir=None), ["v1"])
    with pytest.raises(SystemExit):                         # requires explicit --video
        gh._resolve_detect_from(MagicMock(detect_from=["a"], video=None, videos_dir="d"),
["v1"])
    abs = os.path.join(os.sep, "abs", "p.mp4")
    assert gh._resolve_detect_from(
        MagicMock(detect_from=[abs], video=["v1"], videos_dir=None), ["v1"]) == [abs]
```

33. assistant (2026-06-17T14:36:18.099Z)

Excellent! Now I can see the test at line starting with `def
test_batch_threads_watermark_and_padding_into_compiler`:

```python
def test_batch_threads_watermark_and_padding_into_compiler(tmp_path):
    """REGRESSION: the runner must pass stitching.watermark + begin/end_padding to VideoCompiler
    (it previously constructed VideoCompiler bare -> reels shipped un-watermarked)."""
    src = tmp_path / "clip.MP4"
    src.write_bytes(b"src")
    captured, _cvid, out = _run_one(tmp_path, video_paths=[src])

    wm = captured["ctor"]["watermark"]
    assert wm and wm.get("enabled") is True                 # on-by-default mark threaded
through
    assert wm.get("text") == "Generated by Khelsutra.guru"
    assert captured["ctor"]["begin_padding"] == 0.5         # config paddings, not the ctor 1.0
default
    assert captured["ctor"]["end_padding"] == 0.5
```

The test explicitly asserts:
```python
assert captured["ctor"]["end_padding"] == 0.5
```

But the comment says "config paddings, not the ctor 1.0 default" - which confirms the finding is
correct. The test EXPECTS the new behavior to extract from config.json and get 0.5, but it
acknowledges this is different from the old ctor default of 1.0.

However, the critical question is: **Is this intentional or a bug?** Let me look at the commit
message again:

The commit message says:

> "The batch runner (tools/generate_highlights.py) constructed VideoCompiler bare ? `VideoCompiler(local_work_dir).compile_highlights(...)` ? so reels shipped UN-watermarked (watermark defaults to None/off) and used the ctor's begin/end padding (0.5/1.0) instead of config.json `stitching.{begin_padding,end_padding}` (0.5/0.5). Every other compile call site threads stitching config; this one didn't. **Now it reads `stitching.*` dict-compatibly and passes watermark + paddings through** (the on-by-default branding mark)."

The commit message EXPLICITLY STATES this is INTENTIONAL. The change is: **"Every other compile call site threads stitching config; this one didn't."** So the old behavior was considered a bug/inconsistency.

Let me verify this is what the code intends - let me look at other call sites:

34. user (2026-06-17T14:36:18.938Z)

```
Found 5 files
C:\Users\avidu\Projects\badminton-highlight-indexer\tools\generate_highlights.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compiler.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py
C:\Users\avidu\Projects\badminton-highlight-indexer\scripts\run_process_and_stitch.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_watermark.py
```

35. user (2026-06-17T14:36:20.713Z)

```
617-
618-    # Run compiler with the configured stitching paddings + optional branding watermark
619-    output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
620:    compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
621-                             end_padding=stitching.end_padding,
watermark=stitching.watermark)
622-    try:
623-        out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
--
752-        # clip carries the same configured stitching paddings + watermark as a real
753-        # highlight (watermark default-on).
754-        stitching = load_app_config().stitching
755:        compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
756-                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
757-        try:
758-            res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
```

36. assistant (2026-06-17T14:36:24.021Z)

Perfect! I can see that in `backend/main.py`, the VideoCompiler is ALWAYS called with explicit `begin_padding`, `end_padding`, and `watermark` from the config. The batch runner in `generate_highlights.py` was the ONLY place that wasn't doing this.

Now let me check what the actual behavior difference is in practice. The key question: **Is the change from 1.0 to 0.5 end_padding intentional and desired, or is it a bug?**

Looking at the code:
- config.json specifies: `"end_padding": 0.5`
- The commit message says: "Every other compile call site threads stitching config; this one didn't"
- This implies the intent is for the batch runner to use the SAME config as everywhere else

So the question becomes: **Is this change actually breaking in the legacy path where someone doesn't use --detect-from?**

Let me check if there's a way to preserve backward compatibility:

37. user (2026-06-17T14:36:26.799Z)

```
Exit code 1
head: The term 'head' is not recognized as a name of a cmdlet, function, script
file, or executable program.
Check the spelling of the name, or if a path was included, verify that the path is
correct and try again.
```

38. user (2026-06-17T14:36:29.478Z)

```
623214a Add begin_padding logic to stitcher and config to capture the serve perfectly at
the start of each rally
5836e1e Implement configurable end_padding (default 1.0s)
inside VideoCompiler and config.json
to prevent abrupt rally
endings
```

39. assistant (2026-06-17T14:36:30.848Z)

Let me look at commit 5836e1e which introduced configurable end_padding:

40. user (2026-06-17T14:36:32.025Z)

```
commit 5836e1e280cb87d007a8b3f5b4da3e90566ebdfc
Author: Avi Dullu <avi.dullu@gmail.com>
Date:   Mon May 18 20:42:06 2026 +0530

    Implement configurable end_padding (default 1.0s) inside VideoCompiler and config.json to
prevent abrupt rally endings

 backend/stitcher/compiler.py | 8 +++++---
 config.json                  | 3 +++
 run_process_and_stitch.py    | 6 ++++--
 3 files changed, 12 insertions(+), 5 deletions(-)
```

41. user (2026-06-17T14:36:34.598Z)

```
commit 5836e1e280cb87d007a8b3f5b4da3e90566ebdfc
Author: Avi Dullu <avi.dullu@gmail.com>
Date:   Mon May 18 20:42:06 2026 +0530

    Implement configurable end_padding (default 1.0s) inside VideoCompiler and config.json to
prevent abrupt rally endings

diff --git a/config.json b/config.json
index 2240ea8..40e8b49 100644
--- a/config.json
+++ b/config.json
@@ -28,5 +28,8 @@
     "output": {
         "default_highlights_name": "badminton_highlights.mp4",
         "db_name": "badminton_indexer.db"
+    },
+    "stitching": {
+        "end_padding": 1.0
     }
 }
\ No newline at end of file
```

42. assistant (2026-06-17T14:36:36.470Z)

Aha! So when `end_padding` was first introduced in commit 5836e1e, the config.json DEFAULT was `1.0`. Now let me find when it was changed to `0.5`:

43. user (2026-06-17T14:36:38.099Z)

```
Exit code 1
Select-String: A parameter cannot be found that matches parameter name 'B'.
```

44. user (2026-06-17T14:36:40.622Z)

```
  @@ -73,6 +73,14 @@
            "begin_padding": 0.5,
>           "end_padding": 0.5,
            "use_cache": false,
  -         "min_shot_count": 1
        "stitching": {
  +         "begin_padding": 0.5,
>           "end_padding": 0.5,
            "use_cache": false,
            "min_shot_count": 1
  @@ -32,6 +32,6 @@
        "stitching": {
>           "end_padding": 0.5,
            "use_cache": false,
  -         "min_shot_count": 4
        },
        "stitching": {
> -         "end_padding": 1.0,
  -         "use_cache": true,
> +         "end_padding": 0.5,
  +         "use_cache": false,
            "min_shot_count": 4
        },
        "stitching": {
> -         "end_padding": 1.0
> +         "end_padding": 1.0,
  +         "use_cache": true,
  +         "min_shot_count": 4
  +     },
  +     "stitching": {
> +         "end_padding": 1.0
        }
    }
```

45. assistant (2026-06-17T14:36:42.705Z)

Let me find the specific commit that changed it:

46. user (2026-06-17T14:36:47.041Z)

```
Exit code 1
head: The term 'head' is not recognized as a name of a cmdlet, function, script
file, or executable program.
Check the spelling of the name, or if a path was included, verify that the path is
correct and try again.
```

47. user (2026-06-17T14:36:50.520Z)

```
2cd469e revert(fusion): undo the Gate-A served-default flip (#146) ? confirmed served regression
(eval?serve) (#151)
570cf2b feat(fusion): flip Gate-A (held-shuttle guard) to the served default ? config-reversible
(#146)
10864f3 feat(stitcher): configurable branding watermark on highlight reels (#101)
6353bfb fix(gemini): honor indexing.ai_max_workers (was hardcoded 5); ship serial default (#79)
4ca7526 chore(config): ai_model -> gemini-flash-latest (auto-tracking alias) (#78)
c64a714 fix(reliability): bound WSL/provider timeouts, SQLite WAL+busy_timeout, chunk-step
guard, CORS (#69)
```

```
bdfb42f fix(config): make indexing.wasb.* (and the other live knobs) actually take effect (#66)
d19557e feat(indexer): replace AGPL ultralytics YOLO with permissive torchvision + ByteTrack
d1fe171 feat: add TrackNet shuttle-trajectory segmenter (WSL2) + WASB setup
6becb07 feat: persist and log raw YOLO candidate rallies for fine-tuning
08bb178 feat: add AIVideoProvider abstraction with Gemini provider
45cfff8 refactor: rename rally to play_segment globally
5257fd5 feat: pipelined YOLO chunk processing & unified Gemini model config
e9428a5 feat: complete all medium-effort tasks (logging, normalised velocity, parallel chunks,
thread-safe gemini)
c73a8e8 fix: apply 7 review improvements ? frame-skipping, track pruning, CUDA guard, re-encode
handoff, fix excepts/ordering, configurable tracker
b258693 feat: implement Gemini-based rally detection and core video/geometry utility modules
6c9e585 chore: update input video file path and reduce chunk overlap configuration
501dc16 Parallelize chunk processing using ThreadPoolExecutor for massive speedup
db3d92e Add chunked processing for long videos: split into overlapping segments, process each
via Gemini independently, offset and merge results
623214a Add begin_padding logic to stitcher and config to capture the serve perfectly at the
start of each rally
```

48. assistant (2026-06-17T14:36:51.757Z)

Let me check commit 10864f3:

49. user (2026-06-17T14:36:53.736Z)

```
commit 10864f39182ca3f756546efa532cccb6b23989a8
Author: Avi <avi.dullu@gmail.com>
Date:   Fri Jun 12 14:19:55 2026 +0530

    feat(stitcher): configurable branding watermark on highlight reels (#101)

    * feat(stitcher): configurable branding watermark on highlight reels

    Burn an optional, fully-configurable branding overlay into every highlight clip
    during the existing per-clip re-encode (concat stays -c copy, so it's ~free).

    - backend/config/models.py: new WatermarkConfig under stitching.watermark ? ON by
      default ("Generated by Khelsutra.guru"), with an off switch (enabled: false). Text,
      position (4 corners), opacity, margin, font/font_path, and an optional logo PNG are
      all configurable (validated bounds on opacity/font_size_ratio).
    - backend/pipeline/stitcher/compiler.py: VideoCompiler accepts watermark (model or
      dict); _watermark_filter builds a -vf (drawtext via textfile= to avoid escaping pain,
      plus optional logo movie/overlay) applied to each clip. Graceful fallback: if the
      watermarked encode fails (bad font/logo), the clip re-encodes WITHOUT the watermark so
      a branding misconfig never nukes the reel. Missing logo is skipped with a warning.
    - Wired through /api/compile, run_process_and_stitch.py, and the CLI --trim path (the
      trim is now the quickest one-command watermark preview).
    - Docs: README (feature, config block, knobs, trim demo + standalone ffmpeg preview,
      how to disable), config.json, CODE_MAP stitcher note.
    - tests/test_watermark.py: config defaults/off-switch/bounds, filtergraph builder
      (text+logo, positions, font, the configurable string written to textfile), and
      end-to-end wiring with ffmpeg mocked incl. the graceful fallback. ruff-clean.

    * fix(test): _FakeCompiler test double accepts the new watermark kwarg

    The /api/compile endpoint now constructs VideoCompiler(output_dir, watermark=...);
    test_api_endpoints' fake double's __init__ only took (out_dir), causing
    TypeError in the full-suite CI lane. Accept watermark=None + **kwargs.

    ---------

    Co-authored-by: Claude <noreply@anthropic.com>

diff --git a/config.json b/config.json
index cd63450..b72a93a 100644
```

```
--- a/config.json
+++ b/config.json
@@ -73,6 +73,14 @@
         "begin_padding": 0.5,
         "end_padding": 0.5,
         "use_cache": false,
-        "min_shot_count": 1
+        "min_shot_count": 1,
+        "watermark": {
+            "enabled": true,
+            "text": "Generated by Khelsutra.guru",
+            "position": "bottom-right",
+            "opacity": 0.85,
+            "logo_path": "",
+            "font_path": ""
+        }
     }
 }
\ No newline at end of file
```

50. assistant (2026-06-17T14:36:55.838Z)

So in commit 10864f3, when watermark was added, the config.json already had `"end_padding": 0.5`. Let me check which commit changed it from 1.0 to 0.5:

51. user (2026-06-17T14:36:57.658Z)

```
  +++ b/config.json
  @@ -30,8 +30,8 @@
           "db_name": "badminton_indexer.db"
       },
       "stitching": {
> -         "end_padding": 1.0,
  -         "use_cache": true,
  +         "end_padding": 0.5,
  +         "use_cache": false,
           "min_shot_count": 4
       }
  +++ b/config.json
  @@ -30,6 +30,8 @@
           "db_name": "badminton_indexer.db"
       },
       "stitching": {
> -         "end_padding": 1.0
> +         "end_padding": 1.0,
  +         "use_cache": true,
  +         "min_shot_count": 4
       }
   }
  \ No newline at end of file
       "output": {
           "default_highlights_name": "badminton_highlights.mp4",
           "db_name": "badminton_indexer.db"
  +    },
  +    "stitching": {
> +         "end_padding": 1.0
       }
   }
  \ No newline at end of file

  commit 0b79ab61c4584ea7420b537b38c7b74acf266ee0
```

52. assistant (2026-06-17T14:36:59.988Z)

Let me find the exact commit where it changed from 1.0 to 0.5:

53. user (2026-06-17T14:37:01.422Z)

```
commit 2cd469e803261cb182dfbaf1d9e213d875925e65
commit 570cf2b1bc2455bc235a2b6eb0d60a09c28b833a
commit 10864f39182ca3f756546efa532cccb6b23989a8
         "end_padding": 0.5,
commit 6353bfb32b4b123f934db1b3274510d87842eb6b
commit 4ca75269af3eeef8d3fa799702d207832431914a
commit c64a714620031e3bc35acc18073f3f122d1ce7e2
commit bdfb42f5f8d4644ba195aff355d91be3c147c3bb
commit d19557eb88642387a28d5512efe4505658c3be3d
commit d1fe171a9045f5afb89105aa2f9e294596e7de89
commit 6becb071a67646e420cbbd97ce838f1e3642b19b
commit 08bb178667559d2645ba5318e371c2dffea068b5
commit 45cfff8504d85bdaac0a40fd05241e8364efc018
commit 5257fd5dd6675987d53190973b7dc55c4f9c1ec1
commit e9428a52d71d6e4710118e520bac86afc9da437c
commit c73a8e822a9e5b6af057bf4f7affdf4cfa1541db
commit b2586932adc9da50dbd2f4d680cf557835c76838
commit 6c9e585497bc222d10ed5650ff69466102c03d38
commit 501dc16aa904d00eb34c0d56f524aba41820cb00
commit db3d92edc09ee39179c4d6d6659392e5715df95c
commit 623214a23321a26600e61bbc2a68c3321ad5b018
         "end_padding": 0.5,
commit 0898b99e40136aaed1e00c54b6a097a986febb7e
         "end_padding": 0.5,
commit 4dc36583aa1f3b7e035e13722c039c8245e9542e
-        "end_padding": 1.0,
+        "end_padding": 0.5,
commit b9d7e19ba0fc37de2b41411b9a6dadeaa0cb4032
-        "end_padding": 1.0
+        "end_padding": 1.0,
```

54. assistant (2026-06-17T14:37:02.969Z)

Let me check commit 4dc36583:

55. user (2026-06-17T14:37:04.205Z)

```
commit 4dc36583aa1f3b7e035e13722c039c8245e9542e
Author: Avi Dullu <avi.dullu@gmail.com>
Date:   Mon May 18 22:52:23 2026 +0530

    Upgrade Gemini processing: migrate to gemini-2.5-pro, improve prompt for precise rally and
shuttle_exchange_count detection, increase uncompressed file size limit to 2GB, update config
settings, and switch to new test video

 backend/indexer/gemini.py | 82 ++++++++++++++++++++++++++----------------
 config.json               |  4 +--
 run_process_and_stitch.py |  4 +--
 3 files changed, 47 insertions(+), 43 deletions(-)
```

56. user (2026-06-17T14:37:06.996Z)

```
commit 4dc36583aa1f3b7e035e13722c039c8245e9542e
Author: Avi Dullu <avi.dullu@gmail.com>
Date:   Mon May 18 22:52:23 2026 +0530

    Upgrade Gemini processing: migrate to gemini-2.5-pro, improve prompt for precise rally and
shuttle_exchange_count detection, increase uncompressed file size limit to 2GB, update config
settings, and switch to new test video
```

```
diff --git a/config.json b/config.json
index ad01cdc..9928bbb 100644
--- a/config.json
+++ b/config.json
@@ -30,8 +30,8 @@
         "db_name": "badminton_indexer.db"
     },
     "stitching": {
-        "end_padding": 1.0,
-        "use_cache": true,
+        "end_padding": 0.5,
+        "use_cache": false,
         "min_shot_count": 4
     }
 }
\ No newline at end of file
```

57. assistant (2026-06-17T14:37:14.997Z)

Excellent! So commit 4dc36583 (Upgrade Gemini processing) changed the config default from `end_padding: 1.0` to `end_padding: 0.5`. This was an intentional configuration change.

Now here's the CRITICAL finding: **The new code (commit a6adcc5) now makes the batch runner HONOR the config.json setting, whereas the OLD code ignored it and ALWAYS used the VideoCompiler constructor default of 1.0.**

So we have three timepoints:
1. **Commit 5836e1e** (May 18): Introduced `end_padding` with default 1.0 in config
2. **Commit 4dc36583** (May 18, later): Changed config default from 1.0 to 0.5 (intentional Gemini upgrade)
3. **Old code (master before a6adcc5)**: Used VideoCompiler default of 1.0 regardless of config
4. **Commit a6adcc5** (current PR, June 17): Now reads from config.json and uses 0.5

The question is: **Is this a bug or intentional?**

Looking at the commit message for a6adcc5:
> "Every other compile call site threads stitching config; this one didn't. Now it reads `stitching.*` dict-compatibly..."

This makes it clear the intent is to make the batch runner **consistent with other call sites**. Let me verify other call sites DO read the config value of 0.5:

From the earlier grep, backend/main.py line 620-621:
```python
compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
                         end_padding=stitching.end_padding, watermark=stitching.watermark)
```

This reads from `stitching.end_padding`, which is 0.5 in the current config.json.

CONCLUSION:

The finding is **TECHNICALLY CORRECT** - the end_padding value DID change from 1.0 to 0.5 in the legacy path. However, this is **NOT A BUG** - it's intentional and correct. The change brings the batch runner into alignment with the rest of the codebase.

The finding's framing ("This changes the actual segment padding applied to generated reels, making them behaviorally different from the pre-diff code") is technically accurate but misleading, because:
1. The pre-diff code was ALREADY inconsistent with the rest of the codebase
2. The config had been changed to 0.5 in commit 4dc36583
3. The old code was ignoring that config change in the batch runner only
4. The new code CORRECTLY applies the configured value

Let me create the structured output:

58. user (2026-06-17T14:37:20.790Z)

```
Structured output provided successfully
```